explAIn

How models interact with the real world

The model only asks. Your code decides what runs, and with whose rights.

"The model requests. Another system acts."

Ask on every project

1. What is the worst thing each tool can do if the model calls it wrongly?
2. Whose credentials does each tool run with: the user's, or a service account?
3. Which actions can't be undone, and where does the user confirm them?
4. What happens if the same tool call is retried or sent twice?
5. Which tool results carry untrusted text, such as web pages, emails or documents?

Rules of thumb

- Give each tool one narrow job; never expose raw SQL, HTTP or shell to the model.
- Enforce limits in code; a rule that lives only in the prompt is not a control.
- Require confirmation in the interface for anything that spends, sends or deletes.
- Run every tool with the signed-in user's permissions, never an admin account.

Red flags

- ⚠ One tool like run_sql or call_api that can do almost anything
- ⚠ "We told the model not to" offered as the safety plan
- ⚠ API keys, tokens or card numbers appear in prompts or tool arguments
- ⚠ Nobody can say which tool calls ran yesterday, with which arguments

Key terms

Tool call	A structured request from the model: a tool name plus arguments your app may run.
Least privilege	Give each tool only the access its task needs, and nothing more.
Idempotency key	An ID sent with a request so a retry doesn't repeat the action, like a double booking.
Schema validation	Checking tool arguments against declared types and ranges before running.
MCP server	A program that exposes tools and data to apps through a shared, open protocol.

CHECKLIST

Tool design review

- ☐ One narrow job, with a name and description clear enough for the model to choose it correctly
- ☐ Risk level set: read, reversible write, irreversible, or never exposed to the model
- ☐ Arguments typed and validated in code: types, enums, ranges, and that IDs belong to the user
- ☐ Irreversible or costly actions need user confirmation in the interface, not in chat text
- ☐ Runs with the user's scoped permissions; credentials stay in the app, never in context
- ☐ Every call logged with arguments and result; retries carry an idempotency key